Loan Approval Prediction System

Machine Learning Pipeline Documentation

End-to-End Data Science Project Report

By Ritika koirala

1. Executive Summary

This document provides comprehensive documentation of a machine learning project developed to predict loan approval outcomes. The system leverages applicant financial and demographic features to classify whether a loan application will be approved or rejected, enabling financial institutions to automate and standardize decision-making.

The project follows a structured data science pipeline:

- Dataset ingestion and initial inspection
- Data cleaning, preprocessing, and encoding
- Exploratory Data Analysis (EDA) with univariate, categorical, bivariate, and multivariate analysis
- Outlier detection and removal using the IQR method
- Feature engineering and selection
- Model development using Random Forest Classification
- Hyperparameter tuning with GridSearchCV
- Model evaluation and deployment via serialization

The final optimized model achieved 97.4% accuracy on the held-out test set, demonstrating strong predictive performance across both approval and rejection classes.

2. Dataset Description

2.1 Source and Overview

The dataset used is loan_datasey.csv, a tabular dataset containing loan application records. It comprises 4,269 rows and 13 columns, representing individual loan applications with their corresponding approval status.

2.2 Features

Column Name	Data Type	Description
loan_id	int64	Unique identifier for each loan application (dropped before modeling)
no_of_dependents	int64	Number of financial dependents of the applicant
education	object → int64	Applicant's education level (Graduate / Not Graduate)
self_employed	object → int64	Employment type (Yes / No)
income_annum	int64	Annual income of the applicant (in local currency)

Column Name	Data Type	Description
loan_amount	int64	Requested loan amount
loan_term	int64	Duration of the loan in months
cibil_score	int64	Credit bureau score (creditworthiness indicator)
residential_assets_value	int64	Value of residential assets owned
commercial_assets_value	int64	Value of commercial assets owned
luxury_assets_value	int64	Value of luxury assets owned
bank_asset_value	int64	Value of bank / liquid assets
loan_status	object → int64	Target variable: Approved (1) or Rejected (0)

2.3 Descriptive Statistics

Key summary statistics from data.describe() reveal the following about numeric features:

Feature	Mean	Std Dev	Min	Max
income_annum	5,059,124	2,806,840	200,000	~9,900,000
loan_amount	15,133,450	9,043,363	300,000	~39,500,000
cibil_score	~600	~—	300	900
no_of_dependents	2.50	1.70	0	5

3. Data Cleaning

3.1 Missing Values

A null value check was performed across all 13 columns using data.isnull().sum(). The result confirmed zero missing values in the dataset, requiring no imputation strategies.

3.2 Duplicate Records

A duplicate record check was performed using data.duplicated().sum(). The result returned 0 duplicates, confirming the dataset contains only unique loan application records.

3.3 Column Name Normalization

Inspection of data.columns revealed that several column names contained leading whitespace (e.g., ' no_of_dependents', ' education'). This was corrected by stripping whitespace from all column names:

```
data.columns = data.columns.str.strip()
```

This ensured consistent column access throughout the pipeline without key errors.

3.4 Categorical Encoding

Three categorical columns were encoded to numeric integers using scikit-learn's LabelEncoder to make them compatible with ML models:

Column	Original Values	Encoded Values
education	Graduate, Not Graduate	1 (Graduate), 0 (Not Graduate)
self_employed	Yes, No	1 (Yes), 0 (No)
loan_status	Approved, Rejected	1 (Approved), 0 (Rejected)

```
le = LabelEncoder()
data['education'] = le.fit_transform(data['education'])
data['self_employed'] = le.fit_transform(data['self_employed'])
data['loan_status'] = le.fit_transform(data['loan_status'])
```

3.5 Irrelevant Feature Removal

The loan_id column was removed as it is a row identifier with no predictive value. Retaining it would introduce spurious variance into the model.

```
data.drop('loan_id', axis=1, inplace=True)
```

3.6 Outlier Removal

Outliers were detected and removed using the Interquartile Range (IQR) method for the three primary continuous numerical features: income_annum, loan_amount, and cibil_score. For each column:

- Q1 (25th percentile) and Q3 (75th percentile) were computed.
- IQR = Q3 − Q1
- Lower bound = Q1 − 1.5 × IQR; Upper bound = Q3 + 1.5 × IQR
- Records outside these bounds were removed from the dataset.

```
for col in ['income_annum', 'loan_amount', 'cibil_score']:
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1
```

```
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
data = data[(data[col] >= lower) & (data[col] <= upper)]
```

Note: Due to loop scoping, only the bounds for cibil_score were applied to the final filter. This is a known limitation in the current notebook iteration. Applying per-column filters sequentially would more robustly remove outliers from all three features.

4. Exploratory Data Analysis (EDA)

4.1 Univariate Analysis

Distribution plots (histograms with KDE overlays) were generated for the three core continuous features: income_annum, loan_amount, and cibil_score. These visualizations revealed the spread, skewness, and modality of each variable.

- income_annum: Roughly uniform to right-skewed distribution; most applicants earn between 2.7M and 7.7M.
- loan_amount: Wide spread; peaks around 7–15M range reflecting diverse borrowing needs.
- cibil_score: Fairly uniform distribution across the 300–900 range, with no strong central tendency.

Box plots were also generated for each of these features to visually identify outlier positions relative to the IQR.

4.2 Categorical Feature Analysis

Count plots were generated for education and loan_status to examine class distributions:

- education: The dataset contains roughly comparable proportions of Graduate and Not Graduate applicants.
- loan_status: The target variable is moderately imbalanced, with more Approved applications than Rejected, reflecting realistic loan portfolio dynamics.

4.3 Bivariate Analysis

Box plots were created to examine the relationship between the target variable (loan_status) and continuous features:

- income_annum vs. loan_status: Approved applicants tend to have noticeably higher annual income compared to rejected ones, suggesting income is a meaningful discriminator.
- cibil_score vs. loan_status: Approved applicants exhibit substantially higher CIBIL scores. This is the strongest bivariate signal in the dataset.

4.4 Multivariate Analysis

A correlation heatmap was generated using data.corr() with a 'coolwarm' color scheme (12×8 figure). It revealed:

- cibil_score has a strong positive correlation with loan approval.
- income_annum and loan_amount are moderately correlated with each other.
- Asset-based features (residential, commercial, luxury, bank) show varied inter-correlations.

A pair plot was also generated for income_annum, loan_amount, cibil_score, and loan_status colored by target class. This confirmed that cibil_score provides the cleanest class separation.

5. Feature Engineering & Selection

5.1 Feature Matrix and Target

Following EDA, the dataset was split into feature matrix X and target vector y:

```
X = data.drop('loan_status', axis=1)
y = data['loan_status']
```

The final feature set used for modeling (12 features after dropping loan_id and loan_status) includes:

- no_of_dependents, education, self_employed
- income_annum, loan_amount, loan_term
- cibil_score
- residential_assets_value, commercial_assets_value, luxury_assets_value, bank_asset_value

5.2 Feature Importance (Random Forest)

An initial Random Forest model with 100 estimators was trained to implicitly perform feature importance ranking via the ensemble's impurity-based importance scores. While explicit importances were not printed, the model's high accuracy (97.4%) suggests that cibil_score, income_annum, and asset values collectively drive most predictive power.

6. Model Development

6.1 Train-Test Split

The dataset was split into training and testing subsets using an 80/20 ratio with a fixed random seed for reproducibility:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

This yielded approximately 3,415 training samples and 854 test samples.

6.2 Baseline Model — Random Forest

A Random Forest Classifier with 100 estimators was trained as the baseline model. Random Forest was chosen for:

- Robustness to overfitting via ensemble averaging of decision trees.
- Handling mixed numeric and encoded categorical features natively.
- Built-in feature importance estimation.
- Strong out-of-the-box performance on tabular data.

```
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
```

6.3 Hyperparameter Tuning

GridSearchCV with 5-fold cross-validation was used to optimize the model's hyperparameters:

```
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None]
}
grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=5)
grid.fit(X_train, y_train)
model = grid.best_estimator_
```

The search covered 6 parameter combinations, each evaluated over 5 folds (30 total fits). The best estimator was selected based on cross-validated accuracy.

7. Model Evaluation

7.1 Accuracy

The tuned Random Forest model achieved the following performance on the 854-sample test set:

Metric	Value
Test Accuracy	97.42%
Macro Avg Precision	0.97
Macro Avg Recall	0.97
Macro Avg F1-Score	0.97

7.2 Classification Report

Class	Precision	Recall	F1-Score	Support
0 (Rejected)	0.97	0.99	0.98	536
1 (Approved)	0.98	0.95	0.96	318
Macro Avg	0.97	0.97	0.97	854
Weighted Avg	0.97	0.97	0.97	854

The model performs slightly better at identifying rejections (F1 = 0.98) than approvals (F1 = 0.96), which is consistent with the class distribution. The high recall on rejections (0.99) means very few rejected applicants are incorrectly approved a desirable property for financial risk management.

8. Model Deployment

8.1 Serialization

The trained model was serialized using Python's pickle module for persistent storage and future inference:

```
import pickle
pickle.dump(model, open('model.pkl', 'wb'))
```

The model file (model.pkl) can be loaded in any Python environment to make predictions without retraining.

8.2 Inference Pipeline

A sample inference was demonstrated using a manually constructed feature vector. The encoding conventions used were:

- education: Graduate = 1, Not Graduate = 0
- self_employed: Yes = 1, No = 0

```
model = pickle.load(open('model.pkl', 'rb'))
prediction = model.predict(features)
prob = model.predict_proba(features)[0][1]
```

For the sample applicant (Graduate, Self-Employed, 2 dependents, income 2.9M, CIBIL 547, loan 11.2M):

Inference Output	Value
Prediction	Approved ✅
Approval Probability	91.98%

Note: A UserWarning was raised by sklearn indicating that the feature array was passed without column names, while the model was trained with named features. This can be resolved by wrapping the features array in a pandas DataFrame with matching column names before calling predict().

9. Limitations & Recommendations

9.1 Known Issues

- Outlier removal loop bug: The IQR filter in the notebook applies only cibil_score bounds to the final dataset due to variable reassignment inside the loop. Individual filters per column should be applied explicitly.
- Feature name warning: The inference pipeline passes a raw numpy array, which produces a sklearn UserWarning. Wrapping features in a pd.DataFrame with named columns would suppress this.
- Class imbalance: The target variable has more Approved samples than Rejected. Techniques like SMOTE or class_weight='balanced' could further improve recall for the minority class.

9.2 Recommendations for Improvement

- Add cross-validated confusion matrix for more robust evaluation.
- Implement SHAP or permutation-based feature importance for model interpretability.

- Build an API (e.g., Flask or FastAPI) around the serialized model for production deployment.
- Consider pipeline encapsulation using sklearn.pipeline.Pipeline to bundle preprocessing and modeling steps.
- Log experiment parameters and metrics using MLflow for reproducibility.

10. Summary

This project successfully developed and deployed a loan approval prediction system using a Random Forest classifier. The end-to-end pipeline covered data ingestion, cleaning, EDA, feature engineering, model training, hyperparameter optimization, evaluation, and inference. The final model achieves 97.42% accuracy on unseen test data, making it a reliable decision support tool for loan processing workflows.

Pipeline Stage	Key Action	Outcome
Data Ingestion	Loaded 4,269 records, 13 columns	Clean baseline dataset
Data Cleaning	Strip whitespace, encode categoricals, drop ID	Fully numeric, model-ready data
Outlier Removal	IQR method on continuous features	Reduced noise in training data
EDA	Univariate, bivariate, multivariate analysis	CIBIL score identified as top predictor
Modeling	Random Forest + GridSearchCV (cv=5)	Best estimator selected automatically
Evaluation	Accuracy, Precision, Recall, F1	97.42% test accuracy
Deployment	Pickle serialization + inference demo	Model ready for production use